

This report outlines the key findings and insights derived from the assignment.

## 1. Systemic Fragility and Technical Debt

The analysis of the campus food-delivery startup highlighted how easily production ML systems can degrade without strict governance. The breakdown between different features illustrates Boundary Erosion, where distinct logics bleed into one another. Furthermore, the marketing team reading raw output tables demonstrates the danger of Undeclared Consumers, introducing invisible dependencies. The reliance on 14 undocumented shell scripts creates a pipeline jungle that makes debugging difficult. Mitigating this requires a shift towards tools like MLflow as a tracking registry for pipeline structure and transparency.

## 2. Empirical Model Optimization

During hyperparameter tuning, the tracking data showed that increasing the learning rate had an outsized impact on performance. More importantly, analyzing the loss curves (decreasing training loss paired with plateauing, rather than plummeting, validation accuracy) provided evidence that the model was generalizing rather than overfitting. MLflow's parameter logging bridges the gap between training a model and understanding its convergence behavior.

## 3. Data Version Control

The exercise demonstrated the mechanics of dataset state management. The workflow established a baseline dataset of exactly 1800 rows (v1), updated it to 2801 rows using new\_labels.zip (v2), and executed a rollback to the exact v1 state using git checkout and dvc checkout. Code versioning (Git) is insufficient for machine learning. Data is as dynamic as code and treating data changes as immutable, versioned commits is a must. The ability to instantly and perfectly roll back from 2801 to 1800 rows proves that DVC can act as a failsafe in a production environment. If a new data ingestion pipeline introduces data drift or corruption, this rollback capability is what prevents prolonged system outages.

## 4. Reproducibility

Partner A acts as the author (training, tracking via MLflow, versioning via DVC, and registering to Staging), while Partner B acts as a validator, attempting to replicate the results using only clone/checkout commands and the environment definition without communication. If a model cannot be seamlessly rebuilt by a different engineer on a different machine, it is a liability. This exercise enforces the principle that dependencies, environments, and data states must be explicitly documented as code. Achieving metric parity in this scenario guarantees that the model is CI/CD ready and independent of the idiosyncratic setup of its original developer.